arXiv:1703.10135v2 [cs.CL] 6 Apr 2017

TACOTRON: TOWARDS END-TO-END SPEECH SYNTHESIS

Yuxuan Wang*, **RJ Skerry-Ryan***, **Daisy Stanton, Yonghui Wu, Ron J. Weiss**†, **Navdeep Jaitly,**

Zongheng Yang, Ying Xiao*, **Zhifeng Chen, Samy Bengio**†, **Quoc Le, Yannis Agiomyrgiannakis,**

Rob Clark, Rif A. Saurous*

Google, Inc.
{yxwang,rjryan,rif}@google.com

ABSTRACT

A text-to-speech synthesis system typically consists of multiple stages, such as a text analysis frontend, an acoustic model and an audio synthesis module. Building these components often requires extensive domain expertise and may contain brittle design choices. In this paper, we present Tacotron, an end-to-end generative text-to-speech model that synthesizes speech directly from characters. Given <text, audio> pairs, the model can be trained completely from scratch with random initialization. We present several key techniques to make the sequence-to-sequence framework perform well for this challenging task. Tacotron achieves a 3.82 subjective 5-scale mean opinion score on US English, outperforming a production parametric system in terms of naturalness. In addition, since Tacotron generates speech at the frame level, it's substantially faster than sample-level autoregressive methods.

1 INTRODUCTION

Modern text-to-speech (TTS) pipelines are complex (Taylor, 2009). For example, it is common for statistical parametric TTS to have a text frontend extracting various linguistic features, a duration model, an acoustic feature prediction model and a complex signal-processing-based vocoder (Zen et al., 2009; Agiomyrgiannakis, 2015). These components are based on extensive domain expertise and are laborious to design. They are also trained independently, so errors from each component may compound. The complexity of modern TTS designs thus leads to substantial engineering efforts when building a new system.

There are thus many advantages of an integrated end-to-end TTS system that can be trained on <text, audio> pairs with minimal human annotation. First, such a system alleviates the need for laborious feature engineering, which may involve heuristics and brittle design choices. Second, it more easily allows for rich conditioning on various attributes, such as speaker or language, or high-level features like sentiment. This is because conditioning can occur at the very beginning of the model rather than only on certain components. Similarly, adaptation to new data might also be easier. Finally, a single model is likely to be more robust than a multi-stage model where each component's errors can compound. These advantages imply that an end-to-end model could allow us to train on huge amounts of rich, expressive yet often noisy data found in the real world.

TTS is a large-scale inverse problem: a highly compressed source (text) is "decompressed" into audio. Since the same text can correspond to different pronunciations or speaking styles, this is a particularly difficult learning task for an end-to-end model: it must cope with large variations at the signal level for a given input. Moreover, unlike end-to-end speech recognition (Chan et al., 2016)

*These authors really like tacos.
†These authors would prefer sushi.

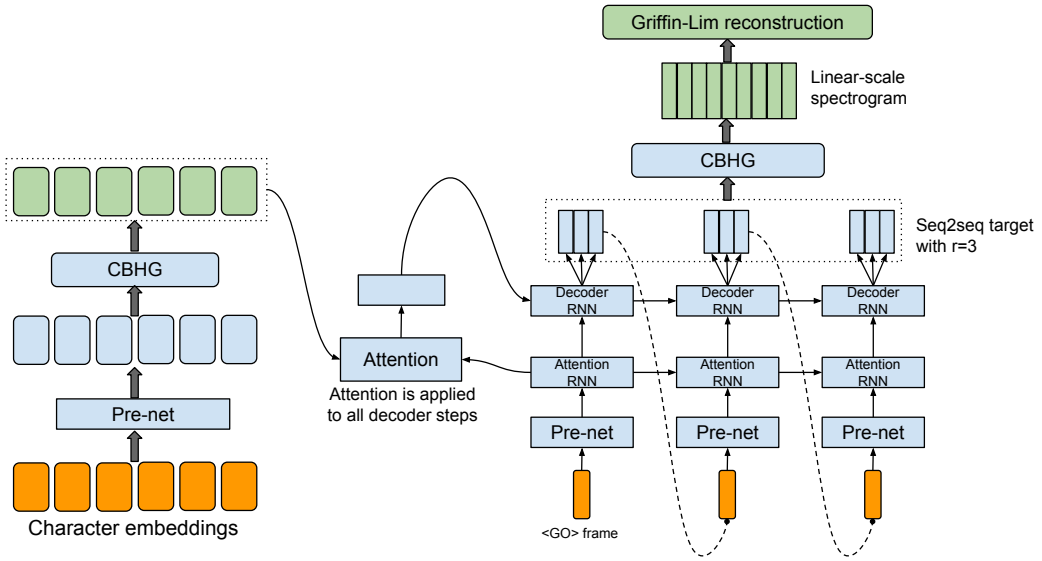


Figure 1: *Model architecture. The model takes characters as input and outputs the corresponding raw spectrogram, which is then fed to the Griffin-Lim reconstruction algorithm to synthesize speech.*

or machine translation (Wu et al., 2016), TTS outputs are continuous, and output sequences are usually much longer than those of the input. These attributes cause prediction errors to accumulate quickly. In this paper, we propose Tacotron, an end-to-end generative TTS model based on the sequence-to-sequence (seq2seq) (Sutskever et al., 2014) with attention paradigm (Bahdanau et al., 2014). Our model takes characters as input and outputs raw spectrogram, using several techniques to improve the capability of a vanilla seq2seq model. Given <text, audio> pairs, Tacotron can be trained completely from scratch with random initialization. It does not require phoneme-level alignment, so it can easily scale to using large amounts of acoustic data with transcripts. With a simple waveform synthesis technique, Tacotron produces a 3.82 mean opinion score (MOS) on an US English eval set, outperforming a production parametric system in terms of naturalness[1].

2 Related Work

WaveNet (van den Oord et al., 2016) is a powerful generative model of audio. It works well for TTS, but is slow due to its sample-level autoregressive nature. It also requires conditioning on linguistic features from an existing TTS frontend, and thus is not end-to-end: it only replaces the vocoder and acoustic model. Another recently-developed neural model is DeepVoice (Arik et al., 2017), which replaces every component in a typical TTS pipeline by a corresponding neural network. However, each component is independently trained, and it's nontrivial to change the system to train in an end-to-end fashion.

To our knowledge, Wang et al. (2016) is the earliest work touching end-to-end TTS using seq2seq with attention. However, it requires a pre-trained hidden Markov model (HMM) aligner to help the seq2seq model learn the alignment. It's hard to tell how much alignment is learned by the seq2seq per se. Second, a few tricks are used to get the model trained, which the authors note hurts prosody. Third, it predicts vocoder parameters hence needs a vocoder. Furthermore, the model is trained on phoneme inputs and the experimental results seem to be somewhat limited.

Char2Wav (Sotelo et al., 2017) is an independently-developed end-to-end model that can be trained on characters. However, Char2Wav still predicts vocoder parameters before using a SampleRNN neural vocoder (Mehri et al., 2016), whereas Tacotron directly predicts raw spectrogram. Also, their seq2seq and SampleRNN models need to be separately pre-trained, but our model can be trained

[1]Sound demos can be found at `https://google.github.io/tacotron`

from scratch. Finally, we made several key modifications to the vanilla seq2seq paradigm. As shown later, a vanilla seq2seq model does not work well for character-level inputs.

3 MODEL ARCHITECTURE

The backbone of Tacotron is a seq2seq model with attention (Bahdanau et al., 2014; Vinyals et al., 2015). Figure 1 depicts the model, which includes an encoder, an attention-based decoder, and a post-processing net. At a high-level, our model takes characters as input and produces spectrogram frames, which are then converted to waveforms. We describe these components below.

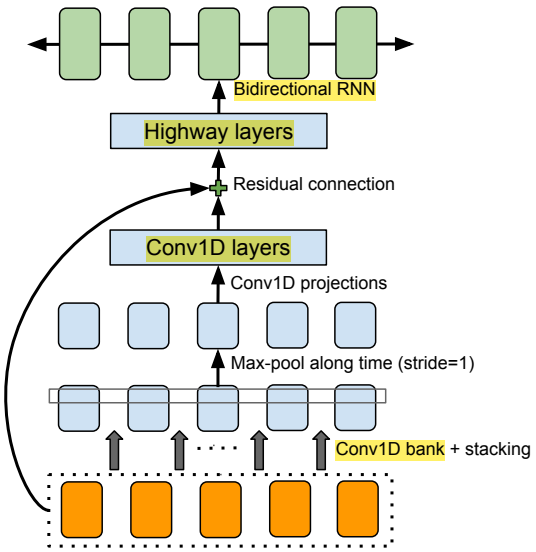


Figure 2: The CBHG (1-D convolution bank + highway network + bidirectional GRU) module adapted from Lee et al. (2016).

3.1 CBHG MODULE

We first describe a building block dubbed CBHG, illustrated in Figure 2. CBHG consists of a bank of 1-D convolutional filters, followed by highway networks (Srivastava et al., 2015) and a bidirectional gated recurrent unit (GRU) (Chung et al., 2014) recurrent neural net (RNN). CBHG is a powerful module for extracting representations from sequences. The input sequence is first convolved with K sets of 1-D convolutional filters, where the k-th set contains C_k filters of width k (i.e. $k = 1, 2, \ldots, K$). These filters explicitly model local and contextual information (akin to modeling unigrams, bigrams, up to K-grams). The convolution outputs are stacked together and further max pooled along time to increase local invariances. Note that we use a stride of 1 to preserve the original time resolution. We further pass the processed sequence to a few fixed-width 1-D convolutions, whose outputs are added with the original input sequence via residual connections (He et al., 2016). Batch normalization (Ioffe & Szegedy, 2015) is used for all convolutional layers. The convolution outputs are fed into a multi-layer highway network to extract high-level features. Finally, we stack a bidirectional GRU RNN on top to extract sequential features from both forward and backward context. CBHG is inspired from work in machine translation (Lee et al., 2016), where the main differences from Lee et al. (2016) include using non-causal convolutions, batch normalization, residual connections, and stride=1 max pooling. We found that these modifications improved generalization.

3.2 ENCODER

The goal of the encoder is to extract robust sequential representations of text. The input to the encoder is a character sequence, where each character is represented as a one-hot vector and em-

Table 1: Hyper-parameters and network architectures. "conv-k-c-ReLU" denotes 1-D convolution with width k and c output channels with ReLU activation. FC stands for fully-connected.

Spectral analysis	*pre-emphasis*: 0.97; *frame length*: 50 ms; *frame shift*: 12.5 ms; *window type*: Hann
Character embedding	256-D
Encoder CBHG	*Conv1D bank*: K=16, conv-k-128-ReLU *Max pooling*: stride=1, width=2 *Conv1D projections*: conv-3-128-ReLU $\rightarrow$ conv-3-128-Linear *Highway net*: 4 layers of FC-128-ReLU *Bidirectional GRU*: 128 cells
Encoder pre-net	FC-256-ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ FC-128-ReLU $\rightarrow$ Dropout(0.5)
Decoder pre-net	FC-256-ReLU $\rightarrow$ Dropout(0.5)$\rightarrow$ FC-128-ReLU $\rightarrow$ Dropout(0.5)
Decoder RNN	2-layer residual GRU (256 cells)
Attention RNN	1-layer GRU (256 cells)
Post-processing net CBHG	*Conv1D bank*: K=8, conv-k-128-ReLU *Max pooling*: stride=1, width=2 *Conv1D projections*: conv-3-256-ReLU $\rightarrow$ conv-3-80-Linear *Highway net*: 4 layers of FC-128-ReLU *Bidirectional GRU*: 128 cells
Reduction factor (r)	2

bedded into a continuous vector. We then apply a set of non-linear transformations, collectively called a "pre-net", to each embedding. We use a bottleneck layer with dropout as the pre-net in this work, which helps convergence and improves generalization. A CBHG module transforms the pre-net outputs into the final encoder representation used by the attention module. We found that this CBHG-based encoder not only reduces overfitting, but also makes fewer mispronunciations than a standard multi-layer RNN encoder (see our linked page of audio samples).

3.3 Decoder

We use a content-based tanh attention decoder (see e.g. Vinyals et al. (2015)), where a stateful recurrent layer produces the attention query at each decoder time step. We concatenate the context vector and the attention RNN cell output to form the input to the decoder RNNs. We use a stack of GRUs with vertical residual connections (Wu et al., 2016) for the decoder. We found the residual connections speed up convergence. The decoder target is an important design choice. While we could directly predict raw spectrogram, it's a highly redundant representation for the purpose of learning alignment between speech signal and text (which is really the motivation of using seq2seq for this task). Because of this redundancy, we use a different target for seq2seq decoding and waveform synthesis. The seq2seq target can be highly compressed as long as it provides sufficient intelligibility and prosody information for an inversion process, which could be fixed or trained. We use 80-band mel-scale spectrogram as the target, though fewer bands or more concise targets such as cepstrum could be used. We use a post-processing network (discussed below) to convert from the seq2seq target to waveform.

We use a simple fully-connected output layer to predict the decoder targets. An important trick we discovered was predicting multiple, non-overlapping output frames at each decoder step. Predicting r frames at once divides the total number of decoder steps by r, which reduces model size, training time, and inference time. More importantly, we found this trick to substantially increase convergence speed, as measured by a much faster (and more stable) alignment learned from attention. This is likely because neighboring speech frames are correlated and each character usually corresponds to multiple frames. Emitting one frame at a time forces the model to attend to the same input token for multiple timesteps; emitting multiple frames allows the attention to move forward early in training. A similar trick is also used in Zen et al. (2016) but mainly to speed up inference.

The first decoder step is conditioned on an all-zero frame, which represents a <GO> frame. In inference, at decoder step t, the last frame of the r predictions is fed as input to the decoder at step $t+1$. Note that feeding the last prediction is an ad-hoc choice here – we could use all r predictions. During training, we always feed every r-th ground truth frame to the decoder. The input frame is passed to a pre-net as is done in the encoder. Since we do not use techniques such as scheduled sampling (Bengio et al., 2015) (we found it to hurt audio quality), the dropout in the pre-net is critical for the model to generalize, as it provides a noise source to resolve the multiple modalities in the output distribution.

3.4 Post-processing net and waveform synthesis

As mentioned above, the post-processing net's task is to convert the seq2seq target to a target that can be synthesized into waveforms. Since we use Griffin-Lim as the synthesizer, the post-processing net learns to predict spectral magnitude sampled on a linear-frequency scale. Another motivation of the post-processing net is that it can see the full decoded sequence. In contrast to seq2seq, which always runs from left to right, it has both forward and backward information to correct the prediction error for each individual frame. In this work, we use a CBHG module for the post-processing net, though a simpler architecture likely works as well. The concept of a post-processing network is highly general. It could be used to predict alternative targets such as vocoder parameters, or as a WaveNet-like neural vocoder (van den Oord et al., 2016; Mehri et al., 2016; Arik et al., 2017) that synthesizes waveform samples directly.

We use the Griffin-Lim algorithm (Griffin & Lim, 1984) to synthesize waveform from the predicted spectrogram. We found that raising the predicted magnitudes by a power of 1.2 before feeding to Griffin-Lim reduces artifacts, likely due to its harmonic enhancement effect. We observed that Griffin-Lim converges after 50 iterations (in fact, about 30 iterations seems to be enough), which is reasonably fast. We implemented Griffin-Lim in TensorFlow (Abadi et al., 2016) hence it's also part of the model. While Griffin-Lim is differentiable (it does not have trainable weights), we do not impose any loss on it in this work. We emphasize that our choice of Griffin-Lim is for simplicity; while it already yields strong results, developing a fast and high-quality trainable spectrogram to waveform inverter is ongoing work.

4 Model Details

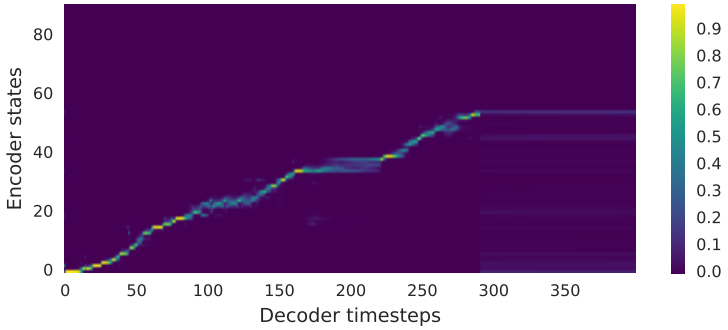
Table 1 lists the hyper-parameters and network architectures. We use log magnitude spectrogram with Hann windowing, 50 ms frame length, 12.5 ms frame shift, and 2048-point Fourier transform. We also found pre-emphasis (0.97) to be helpful. We use 24 kHz sampling rate for all experiments.

We use $r = 2$ (output layer reduction factor) for the MOS results in this paper, though larger r values (e.g. $r = 5$) also work well. We use the Adam optimizer (Kingma & Ba, 2015) with learning rate decay, which starts from 0.001 and is reduced to 0.0005, 0.0003, and 0.0001 after 500K, 1M and 2M global steps, respectively. We use a simple $\ell 1$ loss for both seq2seq decoder (mel-scale spectrogram) and post-processing net (linear-scale spectrogram). The two losses have equal weights.

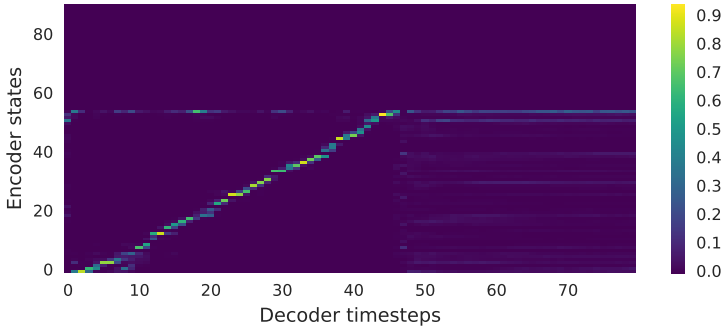
We train using a batch size of 32, where all sequences are padded to a max length. It's a common practice to train sequence models with a loss mask, which masks loss on zero-padded frames. However, we found that models trained this way don't know when to stop emitting outputs, causing repeated sounds towards the end. One simple trick to get around this problem is to also reconstruct the zero-padded frames.

5 Experiments

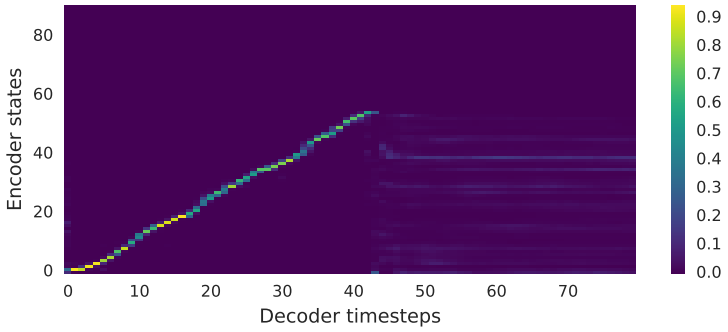
We train Tacotron on an internal North American English dataset, which contains about 24.6 hours of speech data spoken by a professional female speaker. The phrases are text normalized, e.g. "16" is converted to "sixteen".



(a) Vanilla seq2seq + scheduled sampling



(b) GRU encoder



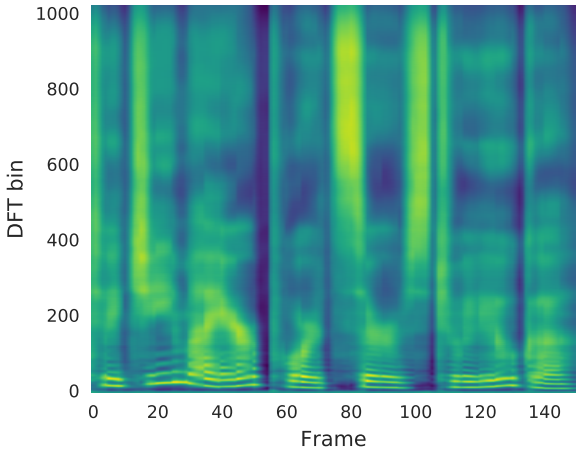
(c) Tacotron (proposed)

Figure 3: *Attention alignments on a test phrase. The decoder length in Tacotron is shorter due to the use of the output reduction factor r=5.*

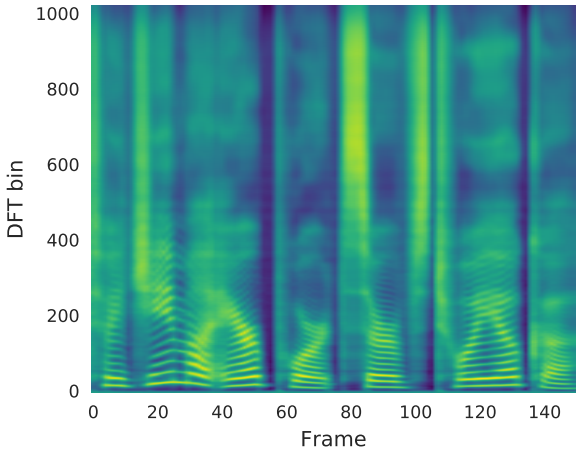
5.1 Ablation Analysis

We conduct a few ablation studies to understand the key components in our model. As is common for generative models, it's hard to compare models based on objective metrics, which often do not correlate well with perception (Theis et al., 2015). We mainly rely on visual comparisons instead. We strongly encourage readers to listen to the provided samples.

First, we compare with a vanilla seq2seq model. Both the encoder and decoder use 2 layers of residual RNNs, where each layer has 256 GRU cells (we tried LSTM and got similar results). No pre-net or post-processing net is used, and the decoder directly predicts linear-scale log magnitude spectrogram. We found that scheduled sampling (sampling rate 0.5) is required for this model to learn alignments and generalize. We show the learned attention alignment in Figure 3. Figure 3(a) reveals that the vanilla seq2seq learns a poor alignment. One problem is that attention tends to



(a) Without post-processing net



(b) With post-processing net

Figure 4: *Predicted spectrograms with and without using the post-processing net.*

get stuck for many frames before moving forward, which causes bad speech intelligibility in the synthesized signal. The naturalness and overall duration are destroyed as a result. In contrast, our model learns a clean and smooth alignment, as shown in Figure 3(c).

Second, we compare with a model with the CBHG encoder replaced by a 2-layer residual GRU encoder. The rest of the model, including the encoder pre-net, remain exactly the same. Comparing Figure 3(b) and 3(c), we can see that the alignment from the GRU encoder is noisier. Listening to synthesized signals, we found that noisy alignment often leads to mispronunciations. The CBHG encoder reduces overfitting and generalizes well to long and complex phrases.

Figures 4(a) and 4(b) demonstrate the benefit of using the post-processing net. We trained a model without the post-processing net while keeping all the other components untouched (except that the decoder RNN predicts linear-scale spectrogram). With more contextual information, the prediction from the post-processing net contains better resolved harmonics (e.g. higher harmonics between bins 100 and 400) and high frequency formant structure, which reduces synthesis artifacts.

5.2 Mean opinion score tests

We conduct mean opinion score tests, where the subjects were asked to rate the naturalness of the stimuli in a 5-point Likert scale score. The MOS tests were crowdsourced from native speakers.

100 unseen phrases were used for the tests and each phrase received 8 ratings. When computing MOS, we only include ratings where headphones were used. We compare our model with a parametric (based on LSTM (Zen et al., 2016)) and a concatenative system (Gonzalvo et al., 2016), both of which are in production. As shown in Table 2, Tacotron achieves an MOS of 3.82, which outperforms the parametric system. Given the strong baselines and the artifacts introduced by the Griffin-Lim synthesis, this represents a very promising result.

Table 2: 5-scale mean opinion score evaluation.

	mean opinion score
Tacotron	3.82 ± 0.085
Parametric	3.69 ± 0.109
Concatenative	4.09 ± 0.119

6 Discussions

We have proposed Tacotron, an integrated end-to-end generative TTS model that takes a character sequence as input and outputs the corresponding spectrogram. With a very simple waveform synthesis module, it achieves a 3.82 MOS score on US English, outperforming a production parametric system in terms of naturalness. Tacotron is frame-based, so the inference is substantially faster than sample-level autoregressive methods. Unlike previous work, Tacotron does not need hand-engineered linguistic features or complex components such as an HMM aligner. It can be trained from scratch with random initialization. We perform simple text normalization, though recent advancements in learned text normalization (Sproat & Jaitly, 2016) may render this unnecessary in the future.

We have yet to investigate many aspects of our model; many early design decisions have gone unchanged. Our output layer, attention module, loss function, and Griffin-Lim-based waveform synthesizer are all ripe for improvement. For example, it's well known that Griffin-Lim outputs may have audible artifacts. We are currently working on fast and high-quality neural-network-based spectrogram inversion.

Acknowledgments

The authors would like to thank Heiga Zen and Ziang Xie for constructive discussions and feedback.

References

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. TensorFlow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.

Yannis Agiomyrgiannakis. Vocaine the vocoder and applications in speech synthesis. In *Acoustics, Speech and Signal Processing (ICASSP), 2015 IEEE International Conference on*, pp. 4230–4234. IEEE, 2015.

Sercan Arik, Mike Chrzanowski, Adam Coates, Gregory Diamos, Andrew Gibiansky, Yongguo Kang, Xian Li, John Miller, Jonathan Raiman, Shubho Sengupta, and Mohammad Shoeybi. Deep voice: Real-time neural text-to-speech. *arXiv preprint arXiv:1702.07825*, 2017.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, pp. 1171–1179, 2015.

William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *Acoustics, Speech and Signal Processing (ICASSP), 2016 IEEE International Conference on*, pp. 4960–4964. IEEE, 2016.

Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.

Xavi Gonzalvo, Siamak Tazari, Chun-an Chan, Markus Becker, Alexander Gutkin, and Hanna Silen. Recent advances in Google real-time HMM-driven unit selection synthesizer. In *Proc. Interspeech*, pp. 2238–2242, 2016.

Daniel Griffin and Jae Lim. Signal estimation from modified short-time fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

Jason Lee, Kyunghyun Cho, and Thomas Hofmann. Fully character-level neural machine translation without explicit segmentation. *arXiv preprint arXiv:1610.03017*, 2016.

Soroush Mehri, Kundan Kumar, Ishaan Gulrajani, Rithesh Kumar, Shubham Jain, Jose Sotelo, Aaron Courville, and Yoshua Bengio. SampleRNN: An unconditional end-to-end neural audio generation model. *arXiv preprint arXiv:1612.07837*, 2016.

Jose Sotelo, Soroush Mehri, Kundan Kumar, João Felipe Santos, Kyle Kastner, Aaron Courville, and Yoshua Bengio. Char2Wav: End-to-end speech synthesis. In *ICLR2017 workshop submission*, 2017.

Richard Sproat and Navdeep Jaitly. RNN approaches to text normalization: A challenge. *arXiv preprint arXiv:1611.00068*, 2016.

Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pp. 3104–3112, 2014.

Paul Taylor. *Text-to-speech synthesis*. Cambridge university press, 2009.

Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. *arXiv preprint arXiv:1511.01844*, 2015.

Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

Oriol Vinyals, Łukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, pp. 2773–2781, 2015.

Wenfu Wang, Shuang Xu, and Bo Xu. First step towards end-to-end parametric TTS synthesis: Generating spectral parameters with neural attention. In *Proceedings Interspeech*, pp. 2243–2247, 2016.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Heiga Zen, Keiichi Tokuda, and Alan W Black. Statistical parametric speech synthesis. *Speech Communication*, 51(11):1039–1064, 2009.

Heiga Zen, Yannis Agiomyrgiannakis, Niels Egberts, Fergus Henderson, and Przemysław Szczepaniak. Fast, compact, and high quality LSTM-RNN based statistical parametric speech synthesizers for mobile devices. *Proceedings Interspeech*, 2016.